

Loguru

Loguru

- **Loguru** is a library which aims to bring enjoyable logging in Python.
- Using **Loguru** you have no excuse not to use logging from the start, this is as simple as **from loguru import logger**.
- Also, this library is intended to make Python logging **less painful** by adding a bunch of **useful functionalities** that solve caveats of the standard loggers.
- **> pip install loguru**
- **Features:**
 - Ready to use out of the box without boilerplate
 - No Handler, no Formatter, no Filter: one function to rule them all
 - Easier file logging with rotation / retention / compression
 - Modern string formatting using braces style
 - Exceptions catching within threads or main
 - Pretty logging with colors
- **Documentation:** <https://loguru.readthedocs.io/en/stable/>

Ready to use out of the box without boilerplate

- For convenience, it is pre-configured and outputs to **stderr** to begin with (but that's entirely configurable).

```
from loguru import logger
```

```
logger.debug("That's it, beautiful and simple logging!")
```

```
logger.error("That's an error!")
```

```
logger.warning("That's a warning!")
```

```
logger.info("That's info!")
```

```
2026-04-20 03:05:25.732 | DEBUG      | __main__:<module>:4 - That's it, beautiful and simple logging!
2026-04-20 03:05:25.734 | ERROR     | __main__:<module>:5 - That's an error!
2026-04-20 03:05:25.736 | WARNING   | __main__:<module>:6 - That's a warning!
2026-04-20 03:05:25.738 | INFO      | __main__:<module>:7 - That's info!
```

Loguru Levels

- Log levels are a widely used concept in logging. They specify the **severity** of a log record so that messages can be filtered or prioritized based on how urgent they are.
- TRACE (5): used to record fine-grained information about the program's execution path for diagnostic purposes.
- DEBUG (10): used by developers to record messages for debugging purposes.
- INFO (20): used to record informational messages that describe the normal operation of the program.
- SUCCESS (25): similar to INFO but used to indicate the success of an operation.
- WARNING (30): used to indicate an unusual event that may require further investigation.
- ERROR (40): used to record error conditions that affected a specific operation.
- CRITICAL (50): used to record error conditions that prevent a core function from working

No Handler, no Formatter, no Filter: one function to rule them all

- How to add a handler? How to set up logs formatting? How to filter messages? How to set level?
- One answer: the [add\(\)](#) function.

```
logger.add(sys.stderr, format="{time} {level} {message}", filter="my_module", level="INFO")
```

- **What is a "sink"?**
- A **sink** is the *destination* for your log messages.
- When you call `logger.add(sink)`, you're telling Loguru: *"send log records here."*
- **If you want to supersede the default stderr handler:**
- just call [logger.remove\(\)](#) to make a fresh start.

Easier file logging with rotation / retention / compression

- If you want to **send logged messages to a file**, you just have to use a string path as the **sink**.
- It can be automatically **timed** too for convenience:

```
logger.add("file_{time}.log")
```

- It is also easily configurable if you need **rotating logger**, if you want to remove older logs, or if you wish to compress your files at closure.

```
logger.add("file_1.log", rotation="500 MB")    # Automatically rotate too big file
logger.add("file_2.log", rotation="12:00")    # New file is created each day at noon
logger.add("file_3.log", rotation="1 week")    # Once the file is too old, it's rotated
logger.add("file_X.log", retention="10 days")  # Cleanup after some time
logger.add("file_Y.log", compression="zip")    # Save some loved space
```

Rotation:

- Automatically **starts a new log file when a condition is met**, so **one file doesn't grow forever**.

```
logger.add("app.log", rotation="10 MB") # new file when size hits 10MB
```

```
logger.add("app.log", rotation="00:00") # new file every midnight
```

```
logger.add("app.log", rotation="1 week") # new file every week
```

Retention:

- Automatically **deletes old log files** so they don't pile up and **fill your disk**.

```
logger.add("app.log", retention="10 days") # delete logs older than 10 days
```

```
logger.add("app.log", retention=3) # keep only the 3 most recent files
```

Compression:

- Automatically **compresses the log file when it's closed (e.g. after rotation)**, **saving disk space**.

```
logger.add("app.log", compression="zip") # compress to .zip
```

```
logger.add("app.log", compression="gz") # compress to .gz
```

Modern string formatting using braces style

- Loguru favors the much more elegant and powerful `{}` formatting over `%`, logging functions are actually equivalent to `str.format()`.

```
logger.info("If you're using Python {}, prefer {feature} of course!", 3.6, feature="f-strings")
```

```
2026-04-20 02:48:20.661 | INFO      | __main__:<module>:4 - If you're using Python 3.6, prefer f-strings of course!
```


Exceptions catching within threads or main

- Have you ever seen your program crashing unexpectedly without seeing anything in the log file?
- This can be solved using the `catch()` decorator / context manager which ensures that any error is correctly propagated to the logger.

@logger.catch

```
def my_function(x, y, z):  
    # An error? It's caught anyway!  
    return 1 / (x + y + z)
```

```
2026-04-20 02:52:46.405 | ERROR      | __main__:<module>:15 - An error has been caught in function '<module>', process 'MainProcess' (13836), thread 'MainThread' (6292):
Traceback (most recent call last):
```

```
> File "H:\TA\ADS(NEW-S26)\LabsPreparation\Lab10-Loguru\code\01_intro.py", line 15, in <module>
    print(my_function(1, -1, 0))
          ^ <function my_function at 0x000001EFD2A20C20>
```

File "H:\TA\ADS(NEW-S26)\LabsPreparation\Lab10-Loguru\code\01_intro.py", line 12, in my_function

```
    return 1 / (x + y + z)
```

Annotations: 1 (under `1`), -1 (under `+`), 0 (under `z`)

```
ZeroDivisionError: division by zero
None
```

Pretty logging with colors

- Loguru **automatically adds colors to your logs** if your terminal is compatible. You can define your favorite style by using [markup tags](#) in the **sink format**.

```
logger.add(sys.stdout, colorize=True, format="<green>{time}</green> <level>{message}</level>")
```

Color/Style Tags — `<green>`, `<level>`

Loguru uses **HTML-like tags** for coloring:

Tag	Meaning
<code><green>...</code> <code></green></code>	Renders text in green
<code><level>...</code> <code></level></code>	Renders text in the color of the log level (e.g. blue for DEBUG, yellow for WARNING, red for ERROR)

Thank You

- <https://betterstack.com/community/guides/logging/loguru/>
- <https://www.youtube.com/watch?v=gSc1oHcwkE4>